购物车 □ 下单核心

gomall 加购 / 地址 / 雪花订单号 / 库存预扣 / DB+outbox 事务

gomall · 业务侧 deck

今日大纲

加购到下单：业务链路的三个停留点

雪花算法：64 bit 怎么切

下单是金融事件：三步必须原子

同步下单 vs 异步下单

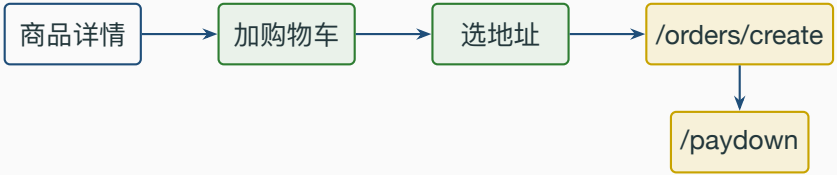
优惠券、拆单、跨地区：业务杂质三连

深挖：雪花、容量、轮询、回滚、订单号、黑产

回顾与代码路径

加购到下单：业务链路的三个停留点

下单不是一次点击，是一条业务链



- 用户视角看着一气呵成，后端是 **4 张表 + 2 个中间件 + 1 条 MQ**。
- 这份 deck 只盯 **加购 / 地址 / 下单**三步；支付走 deck 05 / deck 14。

routes/router.go:92–111 三段路由共同支撑这条链

gomall 购物车的最小数据模型

字段	含义	业务约束
user_id	谁的购物车	索引主键的一部分
boss_id	哪个商家	跨店满减按它聚合
product_id	哪个 SKU	同一 (user, product) 只能一行
num	件数	校验 1.. 库存上限

- `ErrorProductMoreCart=20008`：重复加购同款 □ 给一个明确业务码。
- 这是为什么 `CartCreate` 调用方先 `GetProductById` 再交给 `CreateCart`。

service/cart.go:36–49 + pkg/e/code.go:31–32

地址簿：业务复杂度比想象的高

- **多地址**：家 / 公司 / 父母家，下单时由用户选。
- **默认地址**：新用户首次填的，结算页默认选中。
- **海外地址**：邮编可缺、行政区划不是省/市/区三级。
- **脏数据**：电话 11 位 vs 区号 + 座机，校验放服务端还是前端？

gomall 当前 `Address` 只保留 `name` / `phone` / `address`，
合并字段省事，但订单一旦发起就该**冻结地址快照**，
否则用户改地址后历史订单的收货地址会一起被改。*service/address.go:34–46* 当前模型；
repository/db/model/order.go AddressID 引用关系

为什么订单不用 DB 自增 ID

方案	业务问题	性能问题	
DB AUTO_INCREMENT	暴露当日单量、可枚举	单点瓶颈 / 跨库不连续	**业务侧**
UUID v4	完全随机、无序	索引页频繁分裂	
雪花算法	时间戳前缀，趋势递增	本地生成、零网络	

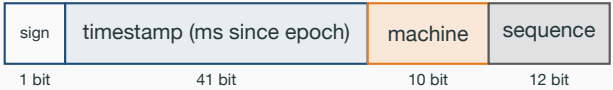
的硬约束：

- 订单号要能塞进短信 / 客服系统，不能太长（雪花 19 位十进制刚好）。
- 不能让竞争对手通过订单号 `id+1` 枚举出当日单量。
- 索引页要顺序写，避免随机 ID 把 B+ 树搅得到处分裂。

pkg/utils/snowflake/snowflake.go:10–25

雪花算法：64 bit 怎么切

雪花 64-bit 结构示意



- **41 bit 时间戳**：从 2024-01-01 起，可用 ≈ 69 年。
- **10 bit 机器位**：最多 1024 个进程，gomall 单实例够用。
- **12 bit 序列号**：单进程单毫秒最多 4096 个 ID。

pkg/utils/snowflake/snowflake.go:11 epoch = 2024-01-01 UTC

InitSnowflake：epoch 与机器位

Listing 1: *pkg/utils/snowflake/snowflake.go:10–21*

```go
func InitSnowflake(machineID int64) {
    snowflake.Epoch = time.Date(
        2024, 1, 1, 0, 0, 0, 0, time.UTC,
    ).UnixNano() / 1000000
    var err error
    node, err = snowflake.NewNode(machineID)
    if err != nil {
        panic(err)
    }
}

func GenSnowflakeID() int64 {
    return node.Generate().Int64()
}
```

逐行讲：为什么这 12 行很要紧

`Epoch=2024-01-01` 把基准从 1970 平移到 2024，腾出最高位的时间空间，41 bit 能多撑几十年。

`machineID` 部署时由配置注入，多副本必须各不相同；冲突 = 双发出现重复订单号。

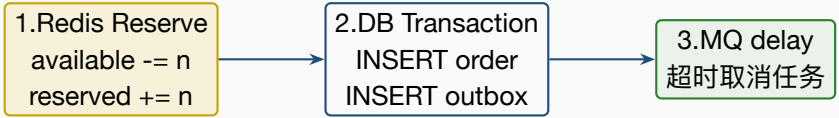
`NewNode` 失败 `panic` 启动期就挂掉，比运行期偶发生成失败更安全——订单号是金融语义，不能”先跑起来再说”。

`Generate().Int64()` 一次返回 int64，方便存 `order.order_num bigint unsigned`。

> 雪花是**本地无锁**的——OrderCreate 调用它零网络成本，这是 50K RPS 的前提。

下单是金融事件：三步必须原子

OrderCreate 拆成三段



- **第 1 步**：Redis Lua 原子 reserve，失败直接 `50001` 不进 DB。
- **第 2 步**：同一 `gorm.Transaction` 写订单 + 写 outbox，二选一不行。
- **第 3 步**：投递延迟队列，15 分钟未付款自动取消。

service/order.go:40–102 全函数 60 行

OrderCreate 主体三步

Listing 2: *service/order.go:46–86*

```go
order := &model.Order{
    UserID: u.Id,
    ProductID: req.ProductID,
    Num: int(req.Num),
    OrderNum: uint64(snowflake.GenSnowflakeID()),
}

// 1) Redis 预扣库存（事务外）
if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
    return nil, err
}

// 2) 同事务写订单 + 写 outbox
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
    if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil {
        return e
    }
    return dao.NewOutboxDaoByDB(tx).Insert(
        "order", "OrderCreated", "order.created", order.ID,
```

逐行讲：为什么 reserve 在 tx 外

- **Redis 不能加入 MySQL 事务**：跨存储两阶段提交业内基本不做，复杂度收益不划算。
- **reserve 失败必须早退**：库存不足 = 业务码 `50001`，不该让 DB 也写一条又回滚。
- **tx 失败要补偿 release**：如果 reserve 成功但 DB 失败，必须把扣下的库存放回 available，否则虚假占用。
- **补偿不是事务的一部分**：即便 `ReleaseReservation` 也失败，DB 已 rollback，库存差额由对账兜底（`GetStockSnapshot` 巡检）。

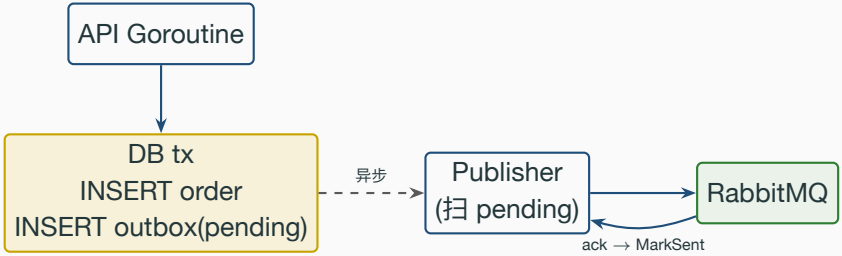
repository/cache/inventory.go:78–125 reserve / commit / release 三脚

逐行讲：为什么 outbox 必须在 tx 内

- 写订单 + 发 MQ 如果**拆成两步**：
  - 写订单成功、发 MQ 前进程挂了 → **下游永远收不到 OrderCreated**。
  - 反过来：先发 MQ 再写 DB，DB 失败 → **幻影订单事件**。
- **outbox 表充当桥梁**：写订单和写 outbox 用同一 `INSERT` 事务，要么都成功要么都没有。
- **异步 publisher** 扫 `status=pending` 的 outbox 行，发 MQ，发完更新成 `sent`。
- 这就是 deck 04 (旧) Outbox 模式在 deck 04 (新) 的落地点。

repository/db/dao/outbox.go:26–41 Insert 同 tx；service/outbox/publisher.go 扫表 publisher

outbox 事件投递时序



业务侧的好处：

API 返回时只承担一次 DB 写代价，MQ 发布失败由 publisher 重试，不阻塞下单链路。

repository/db/dao/outbox.go:44–78 FetchBatch + MarkSent + MarkFailed

同步下单 vs 异步下单

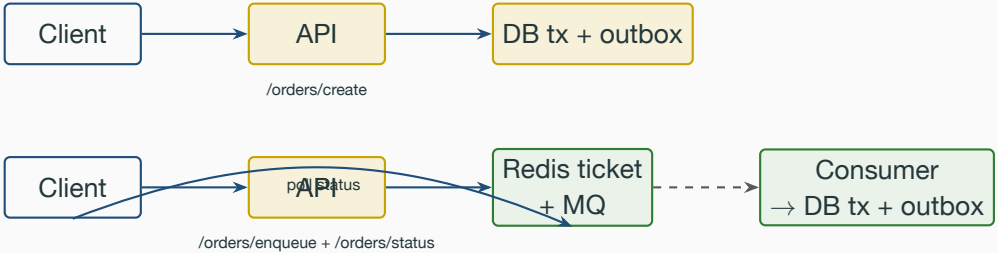
两条下单路并存的业务理由

路径	业务场景	用户体验
`/orders/create`	普通商品 / 流量平稳	一次请求拿到订单号
`/orders/enqueue`	秒杀 / 大促瞬时洪峰	拿 ticket，轮询 status

- 同步：用户等响应的同时 DB 在写；DB 抖动 □ 全部超时。
- 异步：MQ 顶住瞬时压力，consumer 平滑落库；代价是用户多一次轮询。

routes/router.go:92–95 两条路同挂 *Idempotency*

同步 vs 异步：链路对比



service/order_async.go:118–172 enqueue；:175–188 status

OrderEnqueue 核心 12 行

Listing 3: *service/order_async.go:119–150*

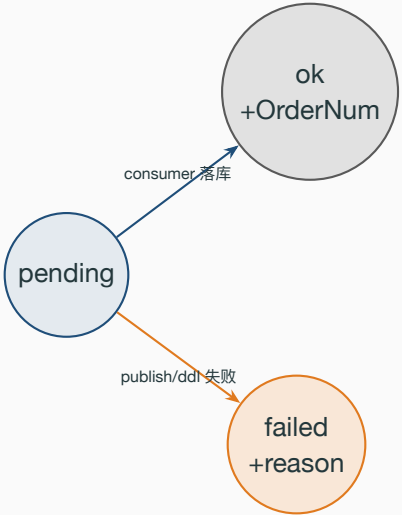
```go
119 func (s *OrderSrv) OrderEnqueue(ctx context.Context,
120     req *types.OrderCreateReq) (resp interface{}, err error) {
121
122     if err = cache.ReserveStock(ctx,
123         req.ProductID, int64(req.Num)); err != nil {
124         return nil, err // 50001 库存不足直接拒
125     }
126
127     ticket := fmt.Sprintf("%d", snowflake.GenSnowflakeID())
128
129     // 把 pending 状态写进 Redis, TTL 1h
130     _ = defaultTicketStore.Put(ctx, ticket,
131         OrderTicketStatus{Status: OrderTicketStatusPending},
132         OrderTicketTTL)
133
134     body, _ := json.Marshal(AsyncOrderTask{Ticket: ticket, ...})
135     return OrderEnqueueResp{Ticket: ticket,
136         Status: OrderTicketStatusPending},
```

逐行讲：ticket 与 TTL 的取舍

- **ticket 用雪花生成**：和订单号同源，全局唯一、客户端不能伪造。
- **TTL = 1h**：覆盖最长支付窗口（15min）+ 客服查单 + 一次客户端崩溃。
- **ticket 不等于订单号**：consumer 落库后回写 `OrderTicketStatus.OrderNum`，客户端读 status 时一并拿到。
- **失败也写 ticket**：投递失败回写 `status=failed, reason=...`，前端不用纠结超时还是真失败。

service/order_async.go:23–52 常量与状态结构

ticket 状态机



- **pending**：API 已收单，consumer 未落库。

下单链路上的业务码

状态码	含义	客户端动作
50001	库存不足	提示”已售罄”，不进重试
60002	幂等命中	用旧响应，不发新单
70001	限流命中	退避后重试
70002	熔断打开	退避后重试（带原 key）

注意：50001 与 60002 表面都是 200，

但含义**完全相反**：
50001 是”业务上不可能成功”，60002 是”已经成功过了”。
客户端必须按 status 字段分支，不能只看 HTTP code。*pkg/e/code.go:49–58* 业务码定义

压测锚点：50,319 RPS / p95 2.33ms

场景	实测	备注
幂等下单 (`/orders/create`) RPS	**50,319**	50 VU × 15s
幂等下单 p95	2.33 ms	同一 `Idempotency-Key` 重
幂等下单 max	73 ms	
实际写入 DB 的订单数	**1**	75 万次请求

放 75 万次，DB 只多 1 行。
说明：业务上的”重复下单防护”成本接近 0，是因为 60002 直接读 Redis done 态。
order 表的体量 6M 行 / 653 MB 不影响这个延迟——不查表就不付代价。

stressTest/REPORT.md § 各链路压测结果 + § 幂等下单 75 万次重放

对账：reserve / commit / release 不变量

- 库存 Redis 双桶：`available + reserved + sold = 初始水位`。
- DB 订单 + outbox 同事务：两条记录要么都在要么都没有。
- ticket + DB 行：`OrderTicketStatus.Status=ok` $\Longleftrightarrow$ `orders.order_num` 存在。

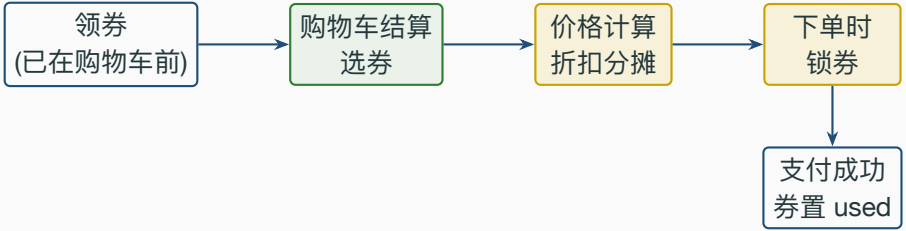
任何一个不变量被破坏，巡检脚本应当告警：

- Redis 双桶不平 → 库存对账脚本（`GetStockSnapshot`）。
- 订单与 outbox 数量不一致 → 直接 SQL JOIN 抽查。
- ticket=ok 但订单不存在 → consumer 写回路径异常。

repository/cache/inventory.go:128–138 GetStockSnapshot 巡检入口

优惠券、拆单、跨地区：业务杂质三连

优惠券进入下单链路的三个关口



- **选券**：用户在结算页主动勾，前端把 `user_coupon.id` 透传到 `/orders/create`。
- **算价**：服务端按（`threshold`，`amount`，`type`）校验，重新算 `money`，不能信前端报价。
- **锁券**：和订单同 tx，把 `user_coupon.status` 从 1 改为 locked，避免一券多用。

repository/db/model/coupon.go:7–14 UserCoupon 状态机；service/coupon.go:64–99 Claim

平台券 vs 店铺券：两条规则同时跑

维度	平台券	店铺券
发行方	gomall 运营	单个 boss_id 店铺
作用域	全站可用	仅该店铺商品
门槛	满 200 减 30	满 100 减 15
叠加	与店铺券可叠加	单店铺只能选一张
分摊	按 SKU 单价占比	整单一次性

业务硬约束：平台券 + 店铺券 + 红包，

三项可同时挂；
但平台券 + 平台券、店铺券 + 同店店铺券 **互斥**，前端禁止勾，服务端二次校验。

repository/db/model/coupon.go:21–32 CouponBatch 字段

下单挂券的伪代码

Listing 4: 下单时挂券的同事务写入示意（*service/order.go* 扩展路径）

```go
err = db.Transaction(func(tx *gorm.DB) error {
   // 1) 锁券 (CAS 防一券多用)
   rows := tx.Model(&UserCoupon{}).
      Where("id = ? AND user_id = ? AND status = ?",
           req.CouponID, u.Id, UserCouponStatusUnused).
      Updates(map[string]any{
         "status": UserCouponStatusLocked,
         "used_order_id": 0, // 先占位
      }).RowsAffected
   if rows == 0 {
      return ErrCouponAlreadyUsed // 70011
   }
   // 2) 用 batch.threshold/amount 重新算 order.money
   if err := recalcOrderMoney(tx, order, batch); err != nil {
      return err
   }
   // 3) 同事务写 order + outbox(OrderCreated)
   return writeOrderAndOutbox(tx, order)
})
```

逐行讲：CAS 防一券多用

`status = Unused` WHERE 加状态条件 = CAS：两个并发请求只有一个能把 1 改成 locked。

`RowsAffected==0` 不是”找不到”，而是**已经被别人锁了**，必须返回业务码而不是 500。

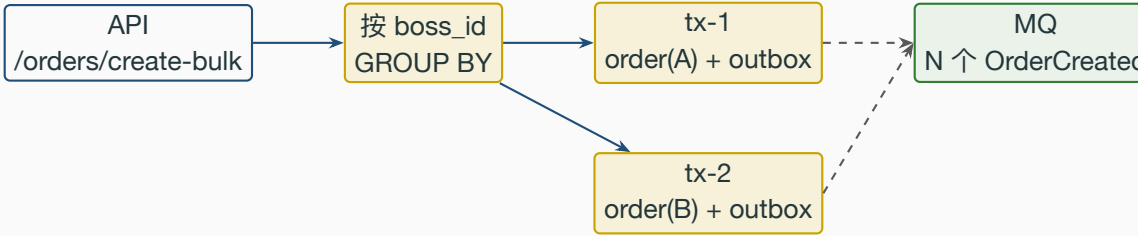
`used_order_id=0` 占位 同 tx 内此时 `order.ID` 还没生成，先占位 0，待 `CreateOrder` 后回填。

`recalcOrderMoney` 校验 `order.money >= batch.threshold`，否则券不可用——**后端校验**比前端报价权威。

`writeOrderAndOutbox` 上面三步任何一步失败都连同订单一起 rollback，券状态自动回 1。

对照 *repository/cache/coupon.go:80–128 ClaimCouponAtomic* 的 *Redis* 路径，*DB* 这里是落库时的最终防线

拆单的时序：一次请求 N 个事务还是 1 个？



- **N 个独立事务**：每个店铺一个 `Transaction`，互相不绑死，A 店事务失败不拖累 B 店。
- **用户视角**：拿到一个父订单号 + N 个子订单号，支付时一次性付完。
- **补偿语义**：父订单 `closed` 时，每个子订单各自走 reserved → available 释放。

routes/router.go:92 单订单路径；拆单批量 *API* 在 *v3* 路线图，本节是设计预演

按 boss_id 分组下单的核心逻辑

Listing 5: 拆单批量下单的服务层主干（*OrderCreateBulk* 设计稿，对照 *service/order.go:40*）

```go
// items 已按 boss_id 排序：A 店 SKU 在前，B 店 SKU 在后
groups := groupByBoss(items)
var children []*model.Order
for _, g := range groups {
   err := db.Transaction(func(tx *gorm.DB) error {
      sub := buildOrder(u.Id, g.BossID, g.Items, parentNum)
      if e := reserveAllSkus(ctx, g.Items); e != nil {
         return e // 50001 直接拒单
      }
      if e := dao.NewOrderDaoByDB(tx).CreateOrder(sub); e != nil {
         return e
      }
      return dao.NewOutboxDaoByDB(tx).Insert(
         "order", "OrderCreated", "order.created", sub.ID,
         events.OrderCreated{OrderID: sub.ID, ParentNum: parentNum, /*...*/},
      )
   })
   if err != nil {
      rollbackReservedForGroups(ctx, groups[:idx]) // 已成功的子订单也要回滚库存
```

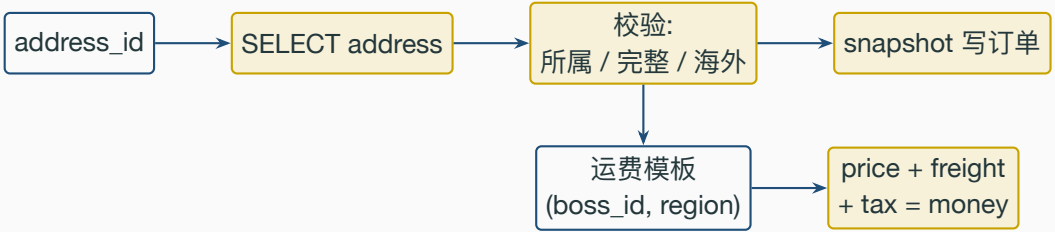
逐行讲：为什么要 ParentNum + 部分回滚

- **ParentNum**：用户视角”一次下单”的逻辑标识；客服查单按它聚合，分账按它对账。
- **逐组 tx 而非一个大 tx**：N 个店铺事务串行，单 tx 大事务会锁太多行 + 跨表锁，吞吐量塌方。
- **部分失败要回滚已扣**：第 3 组失败时，前两组的 Redis `reserved` 必须 release，否则虚假占用持续 15min。
- **outbox 自然多条**：N 个子订单 = N 条 OrderCreated 事件，下游 (发货、对账) 各按子订单消费。

> 拆单是**业务复杂度**问题，不是技术性能问题——数据库吞吐扛得住，但分账规则必须精确到子订单粒度。

对照 *service/order_consumer.go* 现有单订单消费路径，扩展时多条事件并行落库即可

下单时的地址校验链



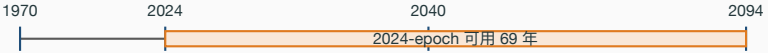
- **所属校验**：`address.user_id == ctx.user_id`，防止 IDOR 借别人地址下单。
- **快照**：把 `name/phone/address/region_code` 写到 order 表，跟用户地址簿解耦。
- **运费/关税**：海外地址额外算关税基数，国内按 (boss_id, 大区) 查模板。

service/address.go:34–46 当前最小模型；下单链路扩展时新增 *RegionCode* + *Snapshot* 字段

深挖：雪花、容量、轮询、回滚、订单号、黑产

为什么 epoch 选 2024-01-01 而不是 1970

- 标准 unix 时间戳 `1970-01-01` 到 2024 已经过去 $\approx$ 54 年 $\approx 1.7 \times 10^{12}$ ms。
- 41 bit 时间戳上限 $\approx 2.2 \times 10^{12}$ ms $\approx$ 69.7 年。
- 如果 epoch 用 1970：**剩余可用 $\approx$ 16 年**，2040 年雪花溢出。
- 如果 epoch 用 2024：**剩余可用 $\approx$ 69 年**，到 2094 年都不用动。



pkg/utils/snowflake/snowflake.go:11 epoch 常量；69 年是 41 bit 上限的精确计算

时钟回拨：业内三种处理方式

- **阻塞等待**：检测到 `now < lastTimestamp` 时，sleep 到追上为止。库内置（bwmarrin/snowflake 默认行为）。
- **扩展位补偿**：保留几个 bit 做”回拨计数器”，发生回拨时 +1，时间戳继续走。Twitter 原版。
- **停机告警**：差距 > 阈值（如 5s）直接拒绝 ID 生成，让 SRE 介入。金融级常用。

- gomall 走第一种——`NewNode` 失败 `panic` 保证启动健康，运行期由库阻塞。
- 配合 NTP `slew` 模式（chronyd 默认）让时钟**平滑追赶**而不是跳变。
- 监控锚点：`GenSnowflakeID p99` ——突然变大就是回拨告警的前哨。

pkg/utils/snowflake/snowflake.go:13–16 panic 策略；*NTP slew vs step* 由运维保证

同步 vs 异步：三档流量下的真实分水岭

QPS 档位	同步 `/orders/create`	异步 `/orders/enqueue`
1k	p95 < 5ms，DB 轻松	多一次轮询，体验劣化
10k	p95 30–50ms，DB 占 70% CPU	p95 < 5ms，consumer 平滑落库
100k	DB 连接池打爆、超时雪崩	MQ 排队、用户最长等 30s 拿订单号

结论：1k 别用异步（体验差）；10k 必须异步（保 DB）；100k 是营销活动级，要预案——**排队页 + 静默降级**。*stressTest/REPORT.md* 实测 *50K RPS* 是幂等读路径，不是真写路径；真写要看 *consumer* 落库节奏

三种轮询方案的取舍

方案	实现复杂度	适用场景
短轮询 (HTTP)	极低	单机部署、状态秒级即可
长轮询 (hold)	中	用户量小、需准实时
SSE (单向流)	中	服务端推送、不需上行
WebSocket	高	双向交互、IM/订单弹幕

gomall 现在用**短轮询**：

500ms / 次共 10 次，理由是单机 goroutine hold 没好处，反而占连接。
扩到多副本 + 用户量 > 10k 时，迁到 SSE 比 WebSocket 划算（只需服务端推 ticket 状态变更，不需上行）。*service/order_async.go:175–188 OrderStatus* 当前为短轮询语义

Redis / DB / outbox 三处失败的责任分摊

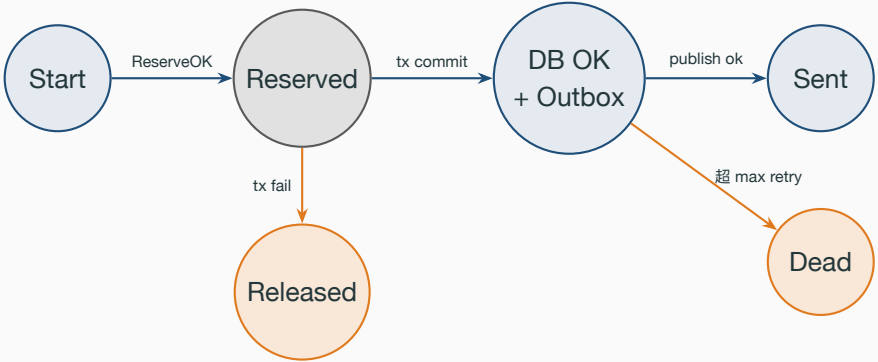
失败点	表现	处理
Redis reserve 失败	50001 库存不足	早退，不进 DB
DB INSERT order 失败	gorm 报错	release reserved
DB INSERT outbox 失败	整 tx rollback	release reserved
publisher 发 MQ 失败	outbox 留 pending	退避重试，5min 内自动恢复
publisher 重试超限	outbox 标 dead	人工补单 / 报警

关键：

每一种失败都有**明确的归属者**，没有” 既不属于 A 也不属于 B” 的灰区。

repository/db/dao/outbox.go:62–78 MarkFailed 退避策略；*service/order.go:79–85* 释放路径

失败回滚状态机



- **Released**：库存退回，订单不存在——客户端看到 `50001`/网络错误，可重试。
- **Dead**：订单已存在但 MQ 没收到——用户能看到订单详情，但下游 (发货、通知) 没动，要人工接管。

repository/db/model/outbox.go OutboxStatusDead 终态；service/outbox/publisher.go 路径

订单号两种主流方案的对照

维度	雪花 19 位纯数字	业务前缀 + 时间 + 自增
样例	1797812345678901234	ORD20260516000001
长度	19 位	18 位
可读性	低	高（一眼看出日期）
枚举性	低（趋势但有跳跃）	高（自增连续）
分库分表	hash 取模即可	必须改成带时间维度的分片
生成成本	本地、零网络	通常要 Redis 自增

gomall 选**雪花纯**

数字：

- 客服那边把订单号前 6 位映射成日期就够用（基于 epoch 反算）；
- 防枚举优先级 > 可读性优先级；
- 零网络生成是 50K RPS 写路径的硬前提。*pkg/utils/snowflake/snowflake.go:19–21 Int64* + 业务上转 *uint64* 落 *order.order_num*

下单链路上的常见黑产手法

- **批量注册 + 单笔下单**：每个新号薅一次” 新人券”，凑齐 100 单合并出货。
- **改 IP + 同设备**：脚本 + 代理池绕过单 IP 限流，但浏览器指纹一致。
- **地址聚合**：100 笔订单收件地址都是同一城中村 / 货代仓库。
- **支付聚合**：100 笔订单背后只有 2–3 张银行卡 / 同一个支付宝。
- **时间聚合**：凌晨 3:00–5:00 集中下单，避开运营值班。

下单服务能给的钩子：把 `ip` / `device_id` / `address_id` / `pay_account` 作为事件字段写到 outbox，
异步推风控系统打分。下单链路本身**不做拦截判断**，否则尾延迟会被风控规则拖垮。

events/order_created.go OrderCreated 已有 UserID/ProductID，扩展加风控字段即可

回顾与代码路径

把这条链拼回业务侧的一句话

- 购物车 = 凑单/比价/留存的中转，不是技术装饰。
- 地址 = 收货语义的快照，下单瞬间必须冻结。
- 雪花订单号 = 业务（防枚举）+ 性能（顺序写）的合谋。
- OrderCreate 三步：**reserve 在事务外、outbox 在事务内**。
- 同步 / 异步两条路：分别面对” 平稳流量” 和” 瞬时洪峰”。

下单是金融事件——gomall 用”Redis 预扣 + DB 事务 + outbox 桥梁”
把它拆成可证不变量的三步。

面试 Q&A（1/3）

Q1：为什么 reserve 不放进 DB 事务?

A：跨 Redis + MySQL 两阶段提交收益不抵复杂度。失败由 ReleaseReservation 补偿，对账兜底。

Q2：outbox 表写满了怎么办?

A：publisher 周期归档 sent；dead 进单独表人工处理。归档脚本按月按 status 分区清。

Q3：雪花时钟回拨在 gomall 怎么处理?

A：库内部阻塞等待，单机部署 NTP 用 slew。监控 GenSnowflakeID p99 作为预警。

Q4：50001 和 60002 客户端怎么区分对待?

A：50001 终止重试、提示售罄；60002 用旧响应跳订单详情，不重发。

面试 Q&A（2/3）

Q5：/orders/enqueue 比 /orders/create 多了什么风险?
A：多一个 ticket 状态需要轮询；consumer 不可用时 ticket 一直 pending 需要 TTL 兜底。

Q6：跨店满减要做，购物车要改什么?
A：聚合算价从 user 维度改为 (user, boss_id) 维度，下单也要拆成多店多单。

Q7：平台券 + 店铺券同时挂，怎么防互斥规则被绕过?
A：服务端按 `batch.scope` 二次校验，前端禁用只是 UX；下单 tx 内 CAS 锁券。

Q8：拆单第 3 个店铺事务失败，前两个怎么办?
A：父订单整体失败，前两个落库的子订单走” 业务侧回滚”——ReleaseReservation + 标 closed。

面试 Q&A（3/3）

Q9：雪花 epoch 选 2024-01-01 之后想再改怎么办？
A：不能动。同表里两种 epoch 解出的 ID 会撞，必须新表新字段或保持 epoch 永久不变。

Q10：100k QPS 营销活动，同步还是异步？
A：异步 + 排队页 + 限流。同步路径留给 1k 量级的” 普通下单”。

Q11：outbox dead 行如何避免越积越多？
A：告警 + 归档 + 人工补单出口；超过 7 天转 outbox_archive 表。

Q12：批量注册薅券下单怎么拦？
A：下单链路不拦，把 ip/device/address/pay 写进 outbox 给风控异步判分；发货前二次决策。

代码位置一览

加购 / 地址 *service/cart.go:29–103；service/address.go:27–127*

雪花 *pkg/utils/snowflake/snowflake.go:10–25*

同步下单 *service/order.go:40–102*

库存预扣/提交/释放 *repository/cache/inventory.go:78–125*

outbox 同事务写入 + 扫表 *repository/db/dao/outbox.go:26–78*

异步下单 + ticket *service/order_async.go:119–188*

优惠券 Redis/DB 两路 *repository/cache/coupon.go ClaimCouponAtomic；repository/db/dao/coupon.go ClaimWithDBLock*

券模型与状态 *repository/db/model/coupon.go:7–32*

OrderCreated 事件 / publisher *service/events/order_created.go；service/outbox/publisher.go*

路由 *routes/router.go:92–120 orders + carts + addresses + coupon*

配套 deck：07 库存 / 08 营销 / 10 流量治理 / 11 一致性。